

Course Outline: Choosing the Right Data Transport Format

Title: Choosing the Right Data Transport Format **Skill:** Skill 40: Cómo optimizar el almacenamiento para reporte, integridad, consulta o transporte **Learnpack:** + ¿Cómo escoger el formato de transporte de los datos? **File:** s40-w14d41-como-escoger-el-formato-de-transporte-de **Prerequisite:** Introducción a Data Pipelines **Target Audience:** Beginner developers who understand data pipelines and ETL **Total Lessons:** 9 **Format:** Conceptual (0% hands-on)

Course Description

Every time data moves between systems — from an API to a frontend, from a pipeline to a database, from a config file to an application — it travels in a format. JSON, CSV, and YAML are the three most common text-based formats in modern software development. Choosing the wrong one causes real problems: parsing failures, data loss, broken integrations, and configuration bugs. This course teaches you when to use each format and how to make the choice deliberately, not by habit.

Course Philosophy

This course emphasizes:

- Decision-making over memorization: the goal is a mental model, not a format spec
 - Real-world consequences: every concept is grounded in what breaks when you choose wrong
 - Clarity over completeness: formats have many features — we focus on the ones that drive the use-case decision
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Why Format Matters	2
02 - The Three Formats	4
03 - Assessment	1
04 - Outro	1
Total	9

00.0 Welcome to Choosing the Right Data Transport Format 

Learning Objectives:

- Understand what this course teaches and why it matters for data pipeline work
- Preview the two sections and the decision framework you'll build

Content Outline:

- The problem: data moves constantly between systems, and every transfer needs a format
- Why format choice is a design decision, not a detail
- What students will be able to do by the end: pick the right format confidently for APIs, data exports, and configuration

Transitions: This course picks up right where the data pipelines course left off. Now that you know how extractors, transformers, and loaders work, this course answers: what format does the data actually travel in?

Key Principles:

- Format choice has upstream and downstream consequences — it affects every system that reads or writes the data
- Three formats handle the majority of real-world use cases: JSON, CSV, and YAML

Examples:

- A FastAPI endpoint returning user data — what format does it use and why?
 - A nightly export of 100,000 sales records — what format makes sense?
 - A Docker Compose file that configures a multi-service application — why is it YAML?
-

01.0 Why Format Matters

Learning Objectives:

- Understand what makes text-based data formats different from one another
- Recognize the real costs of choosing the wrong format
- Identify the three main properties that distinguish formats

Content Outline:

- What "data transport format" means: the agreed-upon shape that data takes while moving between systems
- The three dimensions that separate formats: structure support, human readability, and parsing cost
- What breaks when you choose wrong: concrete failure scenarios
- Overview of Section 01 and what 01.1 will cover

Transitions: This lesson frames the problem. Lesson 01.1 will give you a systematic framework for solving it.

Key Principles:

- Every format is a trade-off between human readability, machine efficiency, and structural expressiveness

- "Default to JSON" is not a decision — it's a habit. Habits cost you when the context doesn't fit
- The format is part of the interface: changing it later breaks every consumer of that data

Examples:

- A team builds an internal API returning nested user objects as CSV. Consumers have to write custom parsers for every nested field — the format doesn't support the data's shape
 - A DevOps engineer stores application config as JSON. Every config change requires checking for missing commas. YAML's comments and indentation would have made this cleaner and safer
 - A data analyst receives pipeline output as YAML. The YAML parser is 40× slower than a CSV reader for 1M rows — wrong format for the volume
-

01.1 Choosing Criteria

Learning Objectives:

- Apply a decision framework to select the appropriate format for a given scenario
- Identify the five key questions that drive format selection
- Map each question to the format it favors

Content Outline:

- The five decision questions:
 1. Is the data flat/tabular or nested/hierarchical?
 2. Who is the primary consumer — a machine or a human?
 3. Does the format need to support comments?
 4. What volume of data will be transported?
 5. What systems need to read or write it?
- How answers map to formats
- The format-to-use-case mental model: API responses → JSON, bulk tabular data → CSV, configuration → YAML
- Preview of Section 02 (each format in depth)

Transitions: You now have the questions to ask. Section 02 gives you the answers — a deep look at each format so you know exactly what each one can and can't do.

Key Principles:

- "What is the data's shape?" is the single most important question — flat data and nested data call for fundamentally different formats
- Comments are a human concern: if humans write or review the file regularly, comment support matters
- Volume and performance matter at scale: CSV parses faster than JSON for flat data; YAML is the slowest of the three

Examples:

- Scenario: you're building a pipeline that exports 500,000 transaction records nightly. Questions: flat or nested? → flat. Machine consumer? → yes. Comments needed? → no. Volume? → high.

Answer: CSV.

- Scenario: a CI/CD system reads a deployment configuration file edited by engineers. Comments needed? → yes. Human readable? → critical. Volume? → tiny. Answer: YAML.
- Scenario: a REST API returns a user object with nested address, preferences, and roles. Nested? → yes. Machine consumer? → yes. Comments? → no. Answer: JSON.

02.0 Data Formats

Learning Objectives:

- Recognize JSON, CSV, and YAML at a glance
- Understand what they share and what fundamentally separates them
- Know which section lessons cover which format in depth

Content Outline:

- What all three formats have in common: text-based, human-readable to some degree, widely supported across languages and tools
- The fundamental differences: structural expressiveness, comment support, parsing speed
- Side-by-side comparison: the same data represented in all three formats
- Preview of 02.1 (JSON), 02.2 (CSV), 02.3 (YAML)

Transitions: This lesson gives you the map. Lessons 02.1 through 02.3 zoom into each format's specific strengths, limits, and ideal use cases.

Key Principles:

- All three formats are text — the differences are structural and semantic, not about encoding
- The same data can be represented in any of the three, but the right format depends on the data's shape and consumer
- No format is universally better — each dominates a specific category of use case

Examples:

- The same contact record shown in JSON (nested, typed), CSV (flat, one row), and YAML (nested, commented):

```
JSON: {"name": "Ana", "email": "ana@example.com", "roles": ["admin",  
"user"]}  
CSV:  name,email,roles\nAna,ana@example.com,"admin,user"  
YAML: name: Ana\n  email: ana@example.com\n  roles:\n    - admin\n    - user
```

- Observation: JSON and YAML handle nesting cleanly; CSV flattens it awkwardly
- Observation: YAML is the most readable for humans; JSON is the most universal for APIs

02.1 JSON

Learning Objectives:

- Describe what JSON is and why it became the dominant API format
- Identify the scenarios where JSON is the correct choice
- Recognize the limits of JSON and the anti-patterns that arise from ignoring them

Content Outline:

- What JSON is: JavaScript Object Notation, a text format for structured data with six value types (string, number, boolean, null, array, object)
- Why JSON dominates API communication: native to JavaScript, supported in every modern language, matches the nested shape of most API payloads
- When to use JSON: REST API responses, event payloads, nested or hierarchical data, inter-service communication
- Strengths: universal support, handles nesting naturally, strongly typed values, compact for structured data
- Limits: no comments, verbose compared to CSV for flat data, not human-friendly for configuration
- Anti-pattern: using JSON for large flat tabular data exports (e.g., 1M rows of sales records) — you pay the structural overhead of JSON for data that has no structure to express

Transitions: JSON is the right choice when data has hierarchy and travels between machines. The next lesson covers CSV — the right choice when data is flat and volume is high.

Key Principles:

- JSON is designed for data, not configuration — the lack of comment support is a deliberate design choice
- Nesting is JSON's superpower: if your data has objects inside objects, JSON expresses it cleanly
- JSON is the default for APIs — if you're building a REST endpoint and have no compelling reason to use another format, JSON is correct

Examples:

- A FastAPI endpoint `/users/{id}` returns a user with nested address and roles — JSON is the right format; the nesting maps directly to the object structure
- A webhook payload from a payment provider contains nested event metadata — JSON captures the hierarchy without custom parsing
- Anti-pattern: a team exports 800,000 product records as JSON for a data analyst. The analyst's pandas job is 3× slower reading JSON than it would be reading CSV, and the file is 40% larger

02.2 CSV

Learning Objectives:

- Describe what CSV is and why it remains the dominant format for flat tabular data
- Identify the scenarios where CSV is the correct choice
- Recognize CSV's structural limitations and the problems they cause

Content Outline:

- What CSV is: Comma-Separated Values, a plain-text format where each row is a record and each column is a field, separated by a delimiter (usually a comma)
- Why CSV dominates bulk data: every spreadsheet application opens it, every database can import/export it, parsers are extremely fast
- When to use CSV: bulk data exports, analytics pipelines, ETL load stages, data sharing between tools (Excel, Google Sheets, pandas, SQL importers)
- Strengths: compact for flat data, universally supported by data tools, fast to parse, easy to inspect in a text editor
- Limits: no nesting (everything must be flat), no native types (all values are strings), delimiter conflicts (commas inside values require quoting), no comments
- Anti-pattern: using CSV for hierarchical data — nesting forces you to either flatten the structure (losing information) or encode nested data as a string inside a field (creating a parsing nightmare)

Transitions: CSV wins when data is flat and volume is high. The next lesson covers YAML — the right choice when humans need to read and write the file directly.

Key Principles:

- CSV is for tabular, flat, high-volume data — if the data fits in a spreadsheet, CSV fits
- Every value in a CSV is a string — any type conversion (dates, numbers, booleans) must happen in the consumer
- The delimiter is a source of bugs: if your data contains commas, you must quote values or use a different delimiter (TSV, pipe-delimited)

Examples:

- A nightly ETL pipeline extracts 500,000 sales transactions and loads them into a data warehouse — CSV is ideal: fast, compact, the warehouse's bulk loader handles it natively
- A data analyst shares a product catalog with a partner — CSV opens directly in Excel without any configuration
- Anti-pattern: a developer stores user profiles with nested address objects in CSV by serializing the address as a JSON string inside the CSV field. The result: `{"city":"Madrid","zip":"28001"}` — now consumers need a JSON parser inside a CSV parser

02.3 YAML

Learning Objectives:

- Describe what YAML is and why it became the standard for human-written configuration
- Identify the scenarios where YAML is the correct choice
- Recognize YAML's critical weakness (indentation sensitivity) and how it causes failures

Content Outline:

- What YAML is: YAML Ain't Markup Language, a human-readable data serialization format designed to be easy for humans to write and read
- Why YAML dominates configuration: comment support, clean indentation-based syntax, no brackets or quotes required for most values
- When to use YAML: CI/CD pipeline definitions (GitHub Actions, GitLab CI), container orchestration (Docker Compose, Kubernetes), application configuration files, any file humans write and maintain
- Strengths: most readable of the three, supports comments, expresses nested structure cleanly without brackets, de facto standard for DevOps tooling
- Limits: indentation-sensitive (a single extra space changes the meaning), slowest parser of the three, not suitable for large data volumes, easy to introduce silent bugs
- Anti-pattern: using YAML as a data transport format between services — indentation bugs are hard to detect, parsers are slow, and YAML's human-friendly features (comments, anchors, aliases) add complexity that machines don't need

Transitions: YAML is the right choice when humans write and maintain the file. With all three formats covered, the next lesson tests your ability to apply the decision framework to realistic scenarios.

Key Principles:

- YAML is for configuration written by humans — if a machine generates the file and another machine reads it, YAML's advantages disappear and its costs remain
- Indentation is syntax in YAML — a misaligned line changes the document's meaning silently, with no error message until the consumer tries to use the value
- Comments are YAML's key differentiator: JSON and CSV cannot be commented; YAML can, making it far more maintainable for complex configuration

Examples:

- A Docker Compose file defines three services with environment variables, port mappings, and volume mounts — YAML's indentation and comment support make this readable and maintainable
- A GitHub Actions workflow defines triggers, jobs, and steps — YAML is the only format GitHub Actions accepts; its nested structure maps cleanly to the workflow graph
- Anti-pattern: a team uses YAML to transfer 10,000 API event records between microservices. The YAML parser takes 8 seconds; the equivalent JSON parse takes 0.3 seconds. The team switches to JSON and eliminates the bottleneck
- Indentation bug example: a Kubernetes deployment YAML where one field is indented two spaces instead of four — the field is silently ignored, the wrong image version gets deployed

03.0 Format Selection Assessment

Learning Objectives:

- Apply the format selection framework to realistic software and data engineering scenarios
- Identify mismatches between format choice and use case
- Explain the consequences of wrong format decisions

Question Format: Scenario-based (select the single best answer)

Questions:

1. A team is building a REST API that returns customer profiles. Each profile contains a nested array of recent orders, and each order has nested line items. Which format should the API use?
 - a) CSV — it is compact and fast to parse
 - b) YAML — it supports comments and nested structures
 - c) JSON — it handles nested structures natively and is the standard for API communication ✓
 - d) Plain text — no format overhead
2. A data engineer needs to export 2 million rows of flat transaction records (transaction_id, amount, date, status) to a data warehouse every night. Which format is most appropriate?
 - a) JSON — it is universally supported
 - b) YAML — it is the most readable
 - c) CSV — it is compact, fast to parse, and ideal for flat tabular data at high volume ✓
 - d) JSON — nested structures make it more expressive
3. A developer is writing a Docker Compose file that will be maintained by five engineers over time. The file defines services, ports, environment variables, and dependencies. Which format should they use?
 - a) JSON — it has universal parser support
 - b) YAML — it supports comments and has the most readable syntax for human-maintained config ✓
 - c) CSV — it is the simplest format
 - d) JSON — it is faster to parse
4. A team stores user configuration objects with nested preferences and role arrays as CSV, serializing nested fields as JSON strings inside CSV cells. What is wrong with this approach?
 - a) Nothing — this is a common and recommended pattern
 - b) CSV cannot store strings, only numbers
 - c) This forces consumers to run a JSON parser inside a CSV parser, creating fragile, hard-to-maintain code ✓
 - d) CSV files cannot be opened in spreadsheet tools
5. A microservice generates 50,000 event records per minute and streams them to another microservice for processing. Both services are owned by the same team. Which format is the right choice?
 - a) YAML — it is the most readable
 - b) CSV — it is compact for flat data, but only if the events are flat ✓ (JSON if events are nested)
 - c) YAML — it is the standard for inter-service communication
 - d) Any format works equally well at this volume

6. An engineer receives a YAML configuration file from a colleague. When the application starts, one setting is silently ignored — it uses the default value instead of the one in the file. What is the most likely cause?
- a) YAML does not support that type of setting
 - b) The YAML parser is outdated
 - c) The field has an indentation error — it is nested under the wrong parent due to a misaligned space ✓
 - d) YAML settings are always ignored on startup
7. A team is choosing a format for a CI/CD pipeline definition that will trigger builds, run tests, and deploy services. The file will be edited regularly by engineers and must be easy to read and annotate. Which format should they use?
- a) CSV — it is the simplest text format
 - b) JSON — it has the widest parser support
 - c) YAML — it supports comments and is the de facto standard for CI/CD configuration ✓
 - d) JSON — it is faster to parse

Evaluation Criteria:

- 7/7: Excellent — you have a solid format selection mental model
 - 5-6/7: Good — review the scenarios you missed and check the decision criteria
 - Below 5: Return to 01.1 (Choosing Criteria) and 02.1-02.3 for the format you're uncertain about
-

04.0 The Right Format for the Job

Content Outline:

- Course recap: what students accomplished
 - Built a decision framework for format selection (the five questions)
 - Learned JSON's strengths (nesting, universal API support) and limits (no comments, verbose for flat data)
 - Learned CSV's strengths (compact, fast, tabular) and limits (no nesting, no types)
 - Learned YAML's strengths (readable, commentable, config-friendly) and limits (indentation-sensitive, slow at scale)
 - Applied the framework to realistic scenarios in the assessment
- The mental model to carry forward:
 - **API response or inter-service message with nested data** → **JSON**
 - **Flat tabular data, bulk export, analytics pipeline** → **CSV**
 - **Human-maintained configuration file** → **YAML**
- Connection to bigger picture: format choice is a recurring decision in every pipeline, every API, and every DevOps workflow — the framework you built here applies everywhere
- What comes next: telemetry and observability — you'll see these formats in action as you learn to collect and transport data from real applications
- Encouragement: format decisions look trivial until they break something. You now have the framework to make them deliberately.

Note: This is conclusion only — no theory, exercises, or assessment.